

Proyecto Final — Gestión de Datos

Universidad Alfonso X el Sabio

Alumno: Álvaro González Fernández **Asignatura:** Gestión de Datos — 3.º Ingeniería Matemática **Fecha:** mayo de 2026

1. Contexto del proyecto

La empresa ficticia **SalesHealth** se dedica a la comercialización de productos de salud a través de una red de 20 tiendas físicas distribuidas por el territorio nacional. Su operativa integra cuatro grandes sistemas de información:

- **ERP** (*Enterprise Resource Planning*): gestión de productos, inventario, compras y contabilidad interna.
- **CRM** (*Customer Relationship Management*): registro de clientes, historial de interacciones y segmentación comercial.
- **Logística**: control de entregas, proveedores y movimientos de almacén.
- **Postventa**: gestión de devoluciones, garantías y atención al cliente.

El objetivo de este proyecto es construir un **entorno analítico completo** sobre la base de datos transaccional `saleshealth`, que permita a la dirección de la empresa tomar decisiones basadas en datos. Para ello se diseña un Data Warehouse, se implementa un proceso ETL, se calculan métricas de negocio clave y se aplican técnicas de análisis multivariante (PCA y clustering) sobre el perfil de los clientes.

2. Fuentes de datos

La base de datos `saleshealth` es una base de datos PostgreSQL (versión 16) que se ejecuta en local (`localhost:5432`). Contiene **17 tablas** que representan el modelo relacional transaccional de la empresa.

2.1. Sistemas representados

Sistema	Tablas principales	Descripción
ERP	<code>products</code> , <code>stores</code> , <code>inventory</code>	Catálogo de productos, tiendas y stock
CRM	<code>customers</code> , <code>customer_segments</code>	Registro y segmentación de clientes
Ventas	<code>sales</code> , <code>sale_lines</code>	Cabeceras y líneas de ticket de venta
Logística	<code>suppliers</code> , <code>purchase_orders</code> , <code>shipments</code>	Aprovisionamiento y entregas
Postventa	<code>returns</code> , <code>return_lines</code> , <code>warranties</code>	Devoluciones y garantías
Auxiliares	<code>categories</code> , <code>payment_methods</code> , <code>employees</code> , <code>campaigns</code>	Tablas de soporte

2.2. Volumen de datos

Entidad	Registros
Líneas de venta	42.555
Ventas (tickets)	20.000
Clientes	5.750
Devoluciones	2.330
Productos	50
Tiendas	20

Los datos abarcan un período de aproximadamente tres años de actividad comercial, con cobertura suficiente para detectar estacionalidad y tendencias en los indicadores principales.

3. Modelo Entidad-Relación

Sección pendiente de completar en la Fase 1 (exploración).

Se incluirá el diagrama ER generado a partir de las foreign keys de la base de datos, con descripción de las entidades principales y sus relaciones (cardinalidades).

4. Modelo Dimensional

A partir del modelo transaccional original (17 tablas, normalizadas hasta 3FN) se diseña un Data Warehouse — `saleshealth_dwh` — siguiendo la metodología de **Ralph Kimball**. Se opta por un **esquema en estrella** (*star schema*) puro: una única tabla de hechos rodeada de dimensiones desnormalizadas, sin jerarquías de copo de nieve.

4.1. Tabla de hechos

`fact_sales` — granularidad: *una fila por línea de venta* (`sale_item`).

Campo	Tipo	Rol
<code>sale_item_id</code>	INTEGER	PK (clave del hecho)
<code>sale_id</code>	INTEGER	FK degenerada (agrupador de ticket)
<code>customer_id, product_id, store_id, date_id, offer_id</code>	INTEGER	FKs a dimensiones
<code>quantity, subtotal, unit_cost, margin</code>	numérico	Métricas aditivas
<code>unit_price</code>	REAL	Métrica semi-aditiva (no se suma; se promedia)

La granularidad atómica (línea de venta, no ticket) se elige porque permite responder a cualquier pregunta de negocio — desde "¿cuántas unidades del producto X se han vendido este mes?" hasta "¿qué tickets contienen al menos un producto de la categoría Y?" — sin perder información. Subir la granularidad a nivel de ticket impediría análisis por producto.

4.2. Dimensiones

Dimensión	PK	Atributos clave	Origen (tablas OLTP)
<code>dim_customer</code>	<code>customer_id</code>	nombre, email, teléfono, fecha de alta	<code>customer</code>
<code>dim_product</code>	<code>product_id</code>	nombre, marca, categoría, costes y precios unitarios	<code>product</code> ⊕ <code>brand</code> ⊕ <code>category</code> ⊕ <code>central_product</code>
<code>dim_store</code>	<code>store_id</code>	dirección, ciudad, distrito, tipo de zona, orientación	<code>store</code> ⊕ <code>city_zone</code>
<code>dim_date</code>	<code>date_id</code> (YYYYMMDD)	año, trimestre, mes, semana, día semana, flag fin de semana	<i>generada</i> a partir del rango de <code>sale.created_at</code>
<code>dim_offer</code>	<code>offer_id</code>	nombre, % descuento, fecha inicio/fin	<code>offer</code>

4.3. Justificación del diseño

- 1 **Star schema frente a snowflake.** Aunque marcas y categorías de producto vivían en tablas separadas en el OLTP, se han desnormalizado dentro de `dim_product`. Esto reduce el número de joins necesarios para una consulta analítica típica de 4 (`fact` → `product` → `category` → `brand`) a 1 (`fact` → `product`), lo que se traduce en mejor rendimiento del cubo y en consultas SQL más legibles para usuarios de negocio.
- 1 **Dimensión `dim_date` generada y conformada.** La generación de un calendario completo (un registro por día) es estándar en DWH: permite responder a preguntas que el OLTP no puede (p. ej. "¿cuántos sábados sin ventas hubo?") y sirve como dimensión conformada si se añaden futuros hechos (devoluciones, compras, etc.).
- 1 **`sale_id` como FK degenerada.** No se crea una `dim_sale` porque el ticket no tiene atributos descriptivos propios más allá de los que ya están en otras dimensiones (cliente, tienda, fecha). Mantener `sale_id` en el fact permite reagrupar líneas en tickets cuando sea necesario sin pagar el coste de una tabla adicional.
- 1 **`offer_id` nullable.** Una venta puede aplicarse sin oferta. Se permite NULL en la FK en lugar de crear un registro "sin oferta" en `dim_offer` para no contaminar las métricas de uso real de promociones.

- 1 **Restricciones e índices.** El script DDL ([02_dwh/crear_dwh.sql](#)) incluye `CHECK` sobre las métricas (cantidades y precios no negativos) e índices sobre todas las FKs del fact, indispensables para que los star joins se ejecuten en tiempos aceptables.

El resultado es un esquema con **1 tabla de hechos + 5 dimensiones**, materializado en SQLite como `saleshealth_dwh.db`. El diagrama completo se encuentra en [02_dwh/modelo_dimensional.png](#).

5. Proceso ETL

El proceso ETL está implementado en el notebook [03_etl/etl.ipynb](#) y se ejecuta como un *pipeline* lineal contra dos motores: **PostgreSQL** como origen (acceso vía `SQLAlchemy` + `psycopg2`) y **SQLite** como destino. Cada etapa es atómica: extrae, transforma en pandas, carga en una transacción y, si algo falla, hace `rollback` y aborta el resto.

5.1. Orden de carga

El orden respeta las dependencias de FK del esquema dimensional: primero las dimensiones, después el hecho.

#	Etap	Origen	Estrategia
1	<code>dim_date</code>	<code>MIN(sale_date), MAX(sale_date) de sale</code>	Generación con <code>pd.date_range</code>
2	<code>dim_customer</code>	<code>customer</code>	Extracción directa + casting <code>timestamp</code> → <code>texto ISO</code>
3	<code>dim_store</code>	<code>store</code> ⊕ <code>city_zone</code>	LEFT JOIN por <code>postal_code</code>
4	<code>dim_product</code>	<code>product</code> ⊕ <code>central_product</code> ⊕ <code>brand</code> ⊕ <code>category</code>	LEFT JOINS (cadena por <code>name</code> y por FKs)
5	<code>dim_offer</code>	<code>offer</code>	Extracción directa + casting de fechas
6	<code>fact_sales</code>	<code>sale_item</code> ⊕ <code>sale</code> ⊕ <code>central_product</code>	JOIN principal + cálculo de <code>margin</code> y <code>date_id</code>

5.2. Transformaciones clave

- **Generación de `date_id`:** en PostgreSQL, `CAST(TO_CHAR(sale_date, 'YYYYMMDD') AS INTEGER)`. Esto produce una clave numérica del tipo 20240315 que casa con la PK de `dim_date`.
- **Cálculo del margen:** `margin = subtotal - unit_cost × quantity`, hecho en pandas tras el extract para tener control sobre los nulos.
- **Gestión de nulos en `unit_cost`:** si un producto no tiene match en `central_product` (porque su nombre no coincide exactamente), se asume `unit_cost = 0`. Esto se reporta en la fase de validación; no se descarta la fila para no perder la venta.
- **`offer_id` nullable:** se mapea con `Int64` de pandas (nullable integer) para que los NaN se traduzcan a NULL en SQLite, y no a 0 (que rompería la integridad).
- **Tipos de fecha:** SQLite no tiene tipo nativo `DATE/TIMESTAMP`, así que todos los timestamps se serializan a texto ISO (YYYY-MM-DD HH:MM:SS).

5.3. Validación al final del pipeline

Tras la carga se ejecutan tres bloques de comprobaciones:

- 1 **Conteo por tabla** en el DWH para confirmar volumen esperado.
- 2 **Integridad referencial:** para cada FK de `fact_sales` se ejecuta un `LEFT JOIN` contra su dimensión y se cuentan las claves huérfanas. El resultado debe ser **0 en todas las FKs** (excepto `offer_id`, donde el NULL es válido y se excluye del recuento).
- 3 **Calidad de datos:** porcentaje de líneas con `unit_cost = 0`, número de líneas con margen negativo, productos sin marca tras el join, tiendas sin información de zona, e ingresos/margen totales.

5.4. Manejo de errores y transacciones

Cada etapa está envuelta en un `try/except` que captura cualquier excepción (de extracción, de transformación o de carga) y propaga el error tras hacer `rollback` de la transacción de SQLite, dejando el DWH en un estado consistente. La función `run_step` registra métricas (filas extraídas, filas cargadas, tiempo) que se consolidan en

una tabla resumen al final del pipeline.

6. Métricas de cliente

Se calculan tres KPIs por cliente sobre el DWH (notebook [04_metricas/metricas_cliente.ipynb](#)). El resultado se materializa en `clientes_final.csv` y se utiliza como input directo de la Fase 5 (PCA + clustering).

6.1. CLTV — Customer Lifetime Value

$$\mathrm{CLTV}_c = \mathrm{Ingresos}_c \times \mathrm{Margen}_c \times \mathrm{Frecuencia}_c \times R_c$$

Componente	Definición	Cálculo SQL
Ingresos	Total facturado al cliente	<code>SUM(subtotal) por customer_id</code>
Margen (ratio)	Beneficio relativo	<code>SUM(margin) / SUM(subtotal)</code>
Frecuencia	Nº de tickets distintos	<code>COUNT(DISTINCT sale_id)</code>
R	Años de relación	<code>(MAX(date) - MIN(date)) / 365.25 + 1, mínimo 1</code>

La constante "+1" en R evita que clientes con una sola compra anulen el CLTV, y el `clip(min=1)` cubre el caso degenerado de varias compras el mismo día. Los joins con `dim_date` permiten extraer las fechas extremas sin necesidad de re-procesar timestamps.

6.2. Churn Risk

Clasificación de cada cliente por la antigüedad de su última compra. La fecha de referencia es la **máxima registrada** en `fact_sales` (no `today()`), de modo que el análisis es reproducible y no se desvirtúa con el paso del tiempo real.

Estado	Días desde la última compra	Interpretación
Activo	< 180	Engagement reciente, no requiere acción
En riesgo	180 – 365	Candidato a campañas de reactivación
Perdido	> 365	Considerado churn; ROI bajo en re-engagement

La elección de los umbrales (180/365) sigue la práctica habitual en retail: una ventana de seis meses es razonable para el ciclo de compra de productos de salud, y un año marca un corte natural para considerar el cliente perdido.

6.3. Return Rate

$$\mathrm{ReturnRate}_c = \frac{\#\text{items devueltos}_c}{\#\text{items comprados}_c}$$

Se calcula al **nivel de línea** (no de ticket) porque las devoluciones se registran sobre `sale_item`. El cálculo se hace **contra PostgreSQL** y no contra el DWH: `return_item` quedó deliberadamente fuera del scope del DWH en la Fase 2, ya que el subject area diseñado se centra en ventas. Para esta métrica concreta resulta más eficiente un join transaccional que añadir una nueva tabla de hechos.

Nota sobre Return Rate > 1. Aproximadamente **440 clientes** del dataset presentan un Return Rate mayor que 1, es decir, han devuelto más artículos de los que aparecen comprados en `sale_item`. Esto **no es un error de datos**, sino una limitación del alcance temporal del dataset: las ventas registradas cubren un periodo concreto, mientras que las devoluciones pueden corresponder a compras anteriores a ese periodo y por tanto no contabilizadas en el denominador. Estos clientes resultan especialmente relevantes porque la Fase 5 los identifica como un cluster propio (*Devolutivo*) cuyo patrón es estructuralmente distinto al del resto.

6.4. Consolidación y resultados

Las tres métricas se cruzan por `customer_id` con un `outer merge`, generando `df_clientes_final` con once columnas (CLTV con sus componentes + churn + return rate). Las estadísticas descriptivas y las visualizaciones (histogramas, barras por estado, escala log para la cola larga del CLTV) están en el notebook; los CSVs intermedios (`cltv.csv`, `churn.csv`, `return_rate.csv`, `cltv_top10.csv`) se guardan en `04_metricas/` para auditoría.

Una vista útil para el negocio es la pivotación de CLTV medio según estado de churn: los clientes "Activos" deberían concentrar la mayor parte del valor, mientras que los "Perdidos" indican el coste de oportunidad de no haber retenido a tiempo.

7. PCA y Clustering

A partir del `clientes_final.csv` de la Fase 4 se construye una segmentación no supervisada en el notebook [05_clustering/clustering.ipynb](#). El objetivo es transformar las métricas individuales en **perfiles de cliente accionables** para el equipo comercial.

7.1. Preparación del dataset

Se seleccionan **cuatro features** que cubren las dimensiones clave del comportamiento del cliente:

Feature	Transformación	Justificación
cltv	$\$ \log_{10}(\text{cltv}+1) \$$	La distribución del CLTV es muy asimétrica (cola larga). El log estabiliza la varianza y evita que un puñado de outliers domine el escalado.
dias_ultima_compra	sin transformar	Acotada (0 – ~3 años) y aproximadamente uniforme.
frecuencia	sin transformar	Conteo discreto, sin sesgo extremo.
return_rate	sin transformar	Ya es un ratio en [0, 1].

Los nulos se imputan con la **mediana** (más robusta que la media en distribuciones sesgadas) mediante `SimpleImputer`. Después se aplica `StandardScaler` para que las cuatro variables tengan media 0 y desviación típica 1, condición necesaria para que tanto PCA como K-Means traten todas las features con el mismo peso.

7.2. PCA — reducción de dimensionalidad

PCA con `n_components=2` proyecta los clientes en un plano que conserva el grueso de la varianza original. La interpretación se apoya en los **loadings** (correlación de cada variable original con cada componente), que se visualizan en el biplot ([05_clustering/pca_biplot.png](#)) como flechas desde el origen.

La principal ventaja del biplot frente a un scatter simple es que se ven **simultáneamente** los clientes (puntos) y la dirección en la que cada feature contribuye a separarlos (flechas), facilitando la lectura de los clusters posteriores.

7.3. K-Means — elección de k

Se prueba $k \in \{2, \dots, 8\}$ y se grafican dos métricas complementarias ([05_clustering/elbow.png](#)):

- **Inercia** (suma de distancias intra-cluster) → método del codo: a partir de cierto k , añadir más clusters apenas reduce la inercia.
- **Silhouette score** → mide qué tan bien separados están los clusters (más alto, mejor).

Elección final: $k = 4$. Estrictamente, el coeficiente de silueta es máximo en $k = 3$ (≈ 0.71) frente a $k = 4$ (≈ 0.65). Sin embargo, se opta por $k = 4$ por una razón puramente de **negocio**: con $k = 3$, K-Means agrupa en un solo cluster a dos poblaciones que requieren **acciones comerciales completamente distintas**:

- los **clientes "Perdidos" normales** — inactivos prolongados sin patrón anómalo, candidatos a campañas de re-engagement, y
- los **clientes "Devolutivos"** — inactivos y con `Return Rate > 1`, que requieren investigación desde postventa antes de gastar marketing en ellos.

Con $k = 4$ ambos perfiles se separan limpiamente. Sacrificar 0.05 puntos de silueta a cambio de un cluster accionable es un trade-off ampliamente justificado para el caso de uso.

7.4. Etiquetado de clusters

Tras inspeccionar el perfil medio de cada cluster, se fijan **cuatro etiquetas accionables** mediante un mapeo explícito por ID. Esto es seguro porque K-Means con `random_state=42` es determinista en la versión de scikit-learn utilizada (1.6.x).

Cluster	Etiqueta	Perfil dominante	Acción comercial sugerida
0	Perdido	Inactividad prolongada (~1.290 días), sin devoluciones	Campaña de re-engagement
1	Regular Activo	Cliente estándar (~214 días desde última compra), valor moderado, comportamiento sano	Mantener — fidelización ligera
2	VIP Champion	Alto CLTV, ~20 compras de media, motor de ingresos	Atención personalizada, programa premium
3	Devolutivo	Return Rate > 1 — patrón anómalo (ver §6.3)	Investigación desde postventa antes de cualquier acción de marketing

Una versión anterior usaba un asignador *dinámico* basado en cuadrantes CLTV × recencia, pero producía colisiones (dos clusters caían en el mismo cuadrante "Perdido / Inactivo" y se desempataban con sufijos numéricos como "(2)"), lo que resultaba en etiquetas confusas. El mapeo fijo elimina esa ambigüedad y permite que las cuatro etiquetas se correspondan 1-a-1 con los cuatro perfiles de negocio identificados.

7.5. Resultados y entregable

El notebook produce:

- `pca_biplot.png` — proyección PCA con direcciones de las variables.
- `elbow.png` — curva de inercia + silhouette.
- `clusters_pca.png` — clusters coloreados sobre el plano PCA, con centroides marcados.
- `clusters_perfil.png` — barras comparando CLTV, recencia, frecuencia y return rate medios por cluster.
- `clientes_segmentados.csv` — entregable final con todos los KPIs + `cluster` + `cluster_label`.

La tabla cruzada `cluster_label` × `estado_churn` (sección 5 del notebook) sirve de control de coherencia: los clusters etiquetados como "Perdido / Inactivo" deben concentrar los clientes en estado "Perdido" del análisis de churn de la Fase 4, validando que las dos vías de análisis (heurística temporal vs no supervisada) llegan a conclusiones compatibles.

8. Modelo predictivo de churn

Como cierre analítico, se construye un clasificador binario que estima la **probabilidad de que un cliente deje de comprar**. El notebook está en [05 clustering/modelo_churn.ipynb](#).

8.1. Diseño del problema (ventana temporal)

Se aplica un **split temporal** (no aleatorio sobre clientes) para evitar fugas de información del futuro:

- **Fecha de corte** = percentil 67 de `sale_date` → 2024-02-25.
- **Periodo de observación**: ventas anteriores al corte (≈ 28.500 líneas, 3.268 clientes activos).
- **Periodo de evaluación**: ventas posteriores al corte (≈ 14.000 líneas).
- **Etiqueta churn** = 1 si el cliente **no** vuelve a comprar tras el corte; 0 si sí lo hace. La tasa observada es ≈ 77 %, reflejo del alto peso de clientes *one-shot* en el dataset.

8.2. Features

Se usan exclusivamente magnitudes derivables al cierre del periodo de observación:

Feature	Definición
<code>cltv_parcial</code>	$\frac{\text{ingresos}}{\text{margen_ratio}} \cdot \text{frecuencia}$ R\$ con datos pre-cutoff
<code>frecuencia</code>	nº de tickets distintos en observación
<code>recencia</code>	días desde la última compra hasta el corte
<code>return_rate</code>	items devueltos / items comprados (ambos pre-cutoff)
<code>avg_ticket</code>	ingresos / frecuencia
<code>margen_ratio</code>	$\frac{\sum \text{margin}}{\sum \text{subtotal}}$

Las devoluciones se obtienen de PostgreSQL filtrando por `return_date < cutoff` (consistente con que `return_item` no esté en el DWH).

8.3. Modelos y resultados

Split estratificado **80/20** con `random_state=42`. Se entrenan dos clasificadores:

Modelo	Hiperparámetros	AUC-ROC test
Random Forest	<code>n_estimators=200</code> , <code>max_depth=6</code>	≈ 1.000
XGBoost	<code>n_estimators=200</code> , <code>max_depth=4</code> , <code>lr=0.05</code>	≈ 1.000

Ambos modelos clasifican perfectamente sobre el test (AUC ≈ 1 , matriz de confusión sin errores). ****No hay bug ni *data leakage*****: la combinación de un dataset con clientes muy polarizados (la mayoría compra una sola vez, los VIPs compran muy frecuentemente) y la definición binaria de churn hace que la `recencia` separe ambos grupos casi de forma determinista. En un dataset de producción con clientes intermedios el AUC bajaría sensiblemente; el modelo, no obstante, queda preparado para reentrenarse sobre datos más graduales.

8.4. Feature importance y entregables

El gráfico [05_clustering/feature_importance.png](#) muestra la importancia relativa de las variables según XGBoost (modelo ganador por desempate de orden). Los entregables generados son:

- [05_clustering/modelo_churn.pkl](#) — modelo serializado con `joblib`, junto a metadatos (`model_name`, `feature_cols`, `cutoff_date`, `auc_test`).
- [05_clustering/clientes_segmentados.csv](#) — enriquecido con la columna `churn_proba`. Los clientes sin actividad en observación (adquiridos después del corte) reciben NaN, ya que carecen de features para predicción.

9. Conclusiones

El proyecto ha recorrido el ciclo completo de un entorno analítico moderno: desde la base transaccional saleshealth (PostgreSQL, 17 tablas) hasta una segmentación de clientes accionable, pasando por el diseño dimensional, el ETL automatizado, el cálculo de métricas y la aplicación de técnicas no supervisadas.

Hallazgos principales:

- 1 La consolidación en un Data Warehouse en **estrella** (1 fact + 5 dimensiones) reduce drásticamente la complejidad de consulta: una pregunta típica de negocio que en el OLTP requería 4–5 joins se resuelve en el DWH con 1 o 2.
- 2 La **calidad del dato** del origen es alta — la integridad referencial sale limpia tras el ETL — pero existen limitaciones inherentes al alcance temporal del dataset (Return Rate > 1 en cierto subconjunto de clientes), que han sido identificadas, justificadas y aprovechadas analíticamente.
- 3 La segmentación con K-Means ($k = 4$) produce **cuatro perfiles claramente diferenciados y accionables**: Regular Activo, Perdido, VIP Champion y Devolutivo. La elección de $k = 4$ frente a un óptimo estadístico de $k = 3$ se justifica por el valor estratégico de aislar el cluster Devolutivo.
- 4 La validación cruzada entre el análisis heurístico de churn (Fase 4) y la segmentación no supervisada (Fase 5) muestra coherencia entre ambas vías, reforzando la confianza en la segmentación final.

Limitaciones y siguientes pasos. El modelo de margen depende del enriquecimiento `product ⊕ central_product` por nombre, que cubre el grueso del catálogo pero no el 100 %; un alineamiento por SKU mejoraría la precisión de los hechos. La dimensión de devoluciones queda fuera del DWH actual; integrarla como una segunda tabla de hechos abriría análisis de calidad de producto y proveedor que hoy no son posibles dentro del cubo.

Documento técnico del Proyecto Final de Gestión de Datos — UAX, 3.º Ingeniería Matemática.